HomeIQ - AI-Powered Smart Home Automation Assistant for Intelligent Device Control

Author: Akhilesh Avinash Patil

1. Project Overview & Objectives

HomeIQ is a powerful, locally-hosted Virtual Assistant designed to operate seamlessly on a pure Python and HTML/CSS architecture. It integrates a conversational AI model capable of general-purpose Q&A, along with specialized tools for managing to-do lists, remembering domain-specific facts via RAG (Retrieval-Augmented Generation), and simulating the state of a smart home.

Core functionalities include:

- Conversational AI: Powered by Groq/HuggingFace via LangChain.
- Smart Home Simulation: Control and view states of simulated locks, lights, blinds, and thermostats.
- Memory Systems: FAISS for vector similarity search of home facts, and Pandas for session-based reminders.

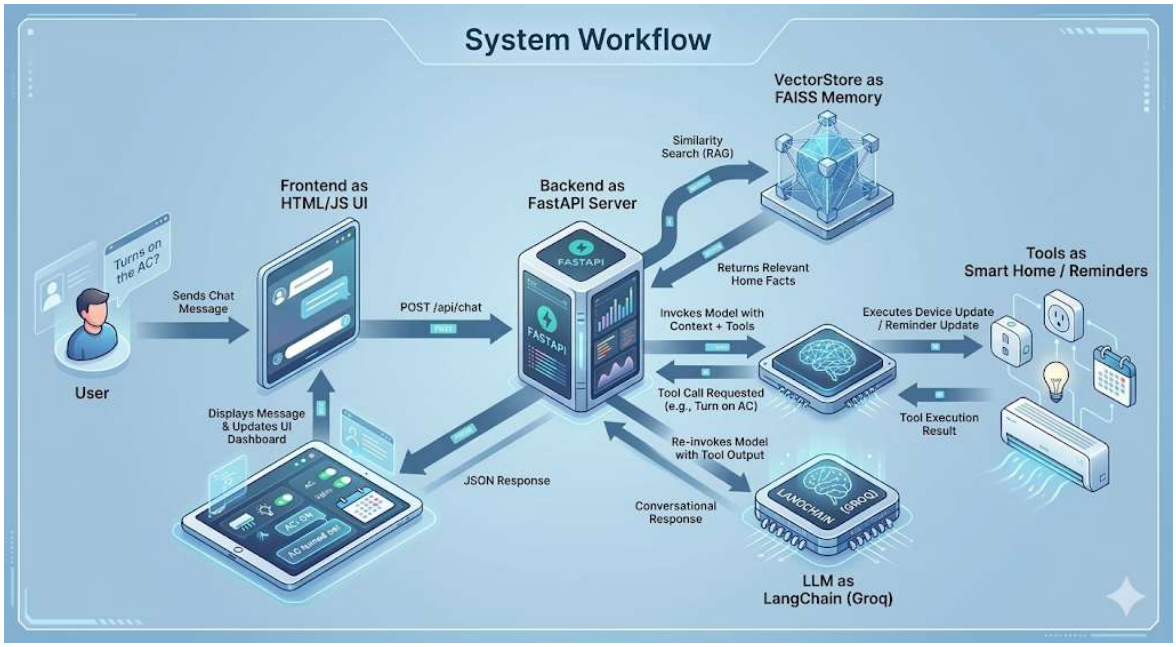
2. Architecture & System Design

The system relies on a unified FastAPI backend that serves both the static frontend files and the REST API. The frontend is built using Vanilla HTML, CSS, and JavaScript, ensuring a lightweight footprint without the need for heavy frameworks like React.

Key AI Components:

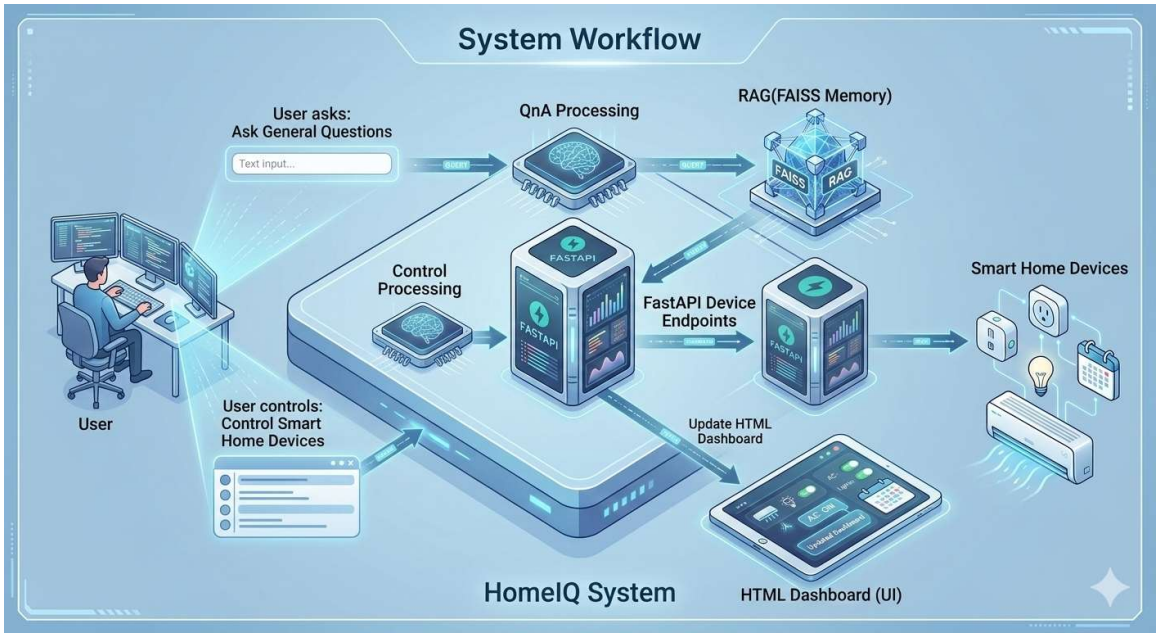
- LLM Orchestration: langchain, langchain-core.
- Vector Store: FAISS local instance via LangChain.
- Embeddings: HuggingFaceEmbeddings (all-MiniLM-L6-v2).

System Workflow



3. Use Case Scenarios

The system supports three primary actors/interactions:



4. Code Structure

```
├── python_backend/
│   ├── main.py (Core FastAPI & LangChain Logic)
│   ├── home_facts.txt (RAG Knowledge Base)
│   ├── .env (API Keys)
│   └── faiss_index/ (Serialized Vector Store)
├── static/
│   ├── index.html (Main UI & Smart home map)
│   ├── style.css (Vanilla styling)
│   └── script.js (Vanilla DOM manipulation and API calls)
├── requirements.txt (Python dependencies)
└── pyproject.toml / uv.lock (uv Package Manager files)
```

5. Setup & Installation

Prerequisites: Python 3.10+, uv or pip.

1. Install Dependencies: Run `uv venv` followed by `uv pip install -r requirements.txt` at the project root.
2. Environment Variables: Create a `.env` file containing `GROQ_API_KEY=your_key`.
3. Start Server: Run `uv run uvicorn main:app --host 0.0.0.0 --port 8000` from the

`python_backend` directory.
4. Access UI: Open `http://localhost:8000`.

6. Core Feature Deep Dive

6.1 Smart Home Simulation

The system maintains a simulated state dictionary for 9 devices, including lights (Havells, Philips), thermostats (Daikin), locks (Godrej), blinds (Wintrobe), and security cameras. The LLM is provided with a `@tool` named `update_device_state` which allows it to naturally mutate this dictionary during conversation. The frontend dynamically polls or refreshes this dictionary to reflect changes visually.

6.2 RAG Knowledge Base

Deep domain knowledge regarding the house's electrical, plumbing, HVAC, and network configurations is stored in `home_facts.txt`. On startup, if a cached FAISS index does not exist, the app chunk-splits this file (1000 chars per chunk) using a HuggingFace embedding model (`all-MiniLM-L6-v2`) and caches it. During chat interactions, the top 3 similarities are retrieved and injected into the LLM prompt.

6.3 Reminder System

A lightweight memory system utilizes the Pandas library to store `id`, `task`, and `created_at` records. The LLM has access to `add_reminder` and `get_reminders` tools to interact with this dataframe.

7. API Endpoints

GET / : Returns the static index.html UI.

GET /api/devices : Returns the current JSON state of all simulated smart devices.

POST /api/chat : Accepts a ChatRequest (message) and returns an LLM-generated ChatResponse.

POST /api/routines/{routine_name} : Executes hardcoded macros (e.g., 'morning', 'night') to bulk-change device states.

8. Conclusion

The **HomeIQ Virtual Assistant** successfully demonstrates how open-source AI models can be transformed from basic chatbots into capable, context-aware smart home managers. By combining a lightweight **FastAPI** backend with **LangChain's** orchestration, **FAISS** for specialized home knowledge retrieval, and simulated tool-calling, the system can understand complex queries and execute tangible device commands. This architecture proves that powerful, responsive AI automation can be built entirely locally without relying on heavy frontend frameworks, resulting in a highly modular and extensible application ready to scale into real-world IoT environments.